Relatório do Projeto

Sistema de Coleta Seletiva

Equipe: Bruno Victor Oliveira de Matos e Monique Andrade de Santana

1. Introdução

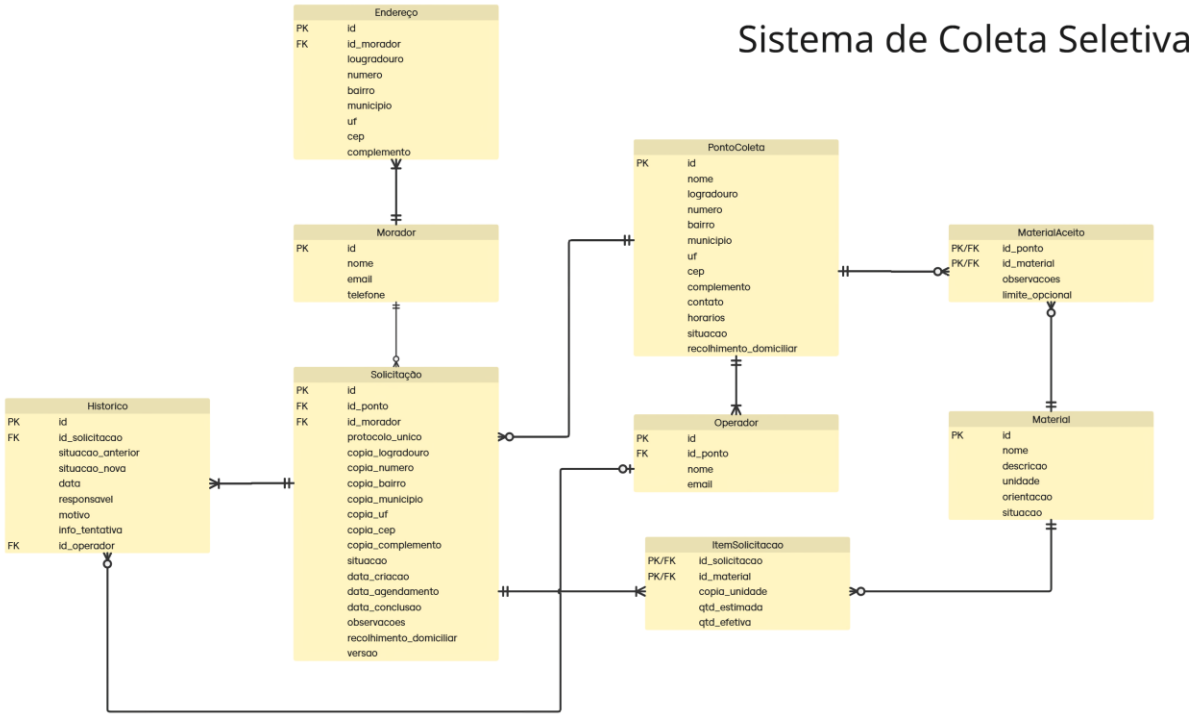
Este relatório documenta o modelo de dados e as decisões de projeto do **Sistema de Coleta Seletiva**, API REST em Spring Boot que conecta moradores, pontos de coleta e operadores para organizar o recolhimento de materiais recicláveis. Ele complementa o README do repositório, que traz as tecnologias e as instruções para executar a aplicação.

O escopo desta entrega é o modelo de dados, a evolução do banco e as decisões tomadas até agora. Os testes automatizados e a interface web ficam para as próximas etapas.

2. Modelo de dados

O banco é o PostgreSQL, com todas as tabelas no schema coleta_seletiva. A estrutura é criada e alterada somente por migrações do Flyway (V1 a V16), e o Hibernate apenas valida o schema (ddl-auto=validate). Os nomes de tabelas e colunas seguem snake_case. As chaves primárias simples são inteiros gerados por sequência. As entidades de associação (materialaceito e itemsolicitacao) usam chave primária composta.

2.1 Diagrama



2.2 Relacionamentos

Entidade A	Cardinalidade	Entidade B	Ligação
`morador`	1:N	`endereco`	`endereco.id_morador`
`morador`	1:N	`solicitacao`	`solicitacao.id_morador`
`pontocoleta`	1:N	`solicitacao`	`solicitacao.id_ponto`
`pontocoleta`	1:N	`operador`	`operador.id_ponto`
`solicitacao`	1:N	`itemsolicitacao`	`itemsolicitacao.id_solicitacao`
`solicitacao`	1:N	`historico`	`historico.id_solicitacao`
`operador`	1:N	`historico`	`historico.id_operador`
`pontocoleta`	N:M	`material`	tabela associativa `materialaceito`
`solicitacao`	N:M	`material`	tabela associativa `itemsolicitacao`

Um morador tem vários endereços e faz várias solicitações. Cada solicitação é atendida por um único ponto, reúne vários itens (um por material) e acumula um histórico de mudanças de situação. Cada ponto aceita vários materiais e tem vários operadores, mas cada operador pertence a um só ponto e não existe sem ele. O vínculo entre histórico e operador é opcional: quando está vazio, a ação foi do morador (ver 4.6).

2.3 Dicionário de dados

Situações (guardadas como texto, validadas por enum na aplicação)

Enum	Valores
`SituacaoMaterial`, `SituacaoPonto`	`ATIVO`, `INATIVO`
`SituacaoSolicitacao`	`SOLICITADA`, `AGENDADA`, `EM_RECOLHIMENTO`, `CONCLUIDA`, `RECUSADA`, `CANCELADA`

morador

Coluna	Tipo	Nulo	Observações
`id`	integer	Não	PK
`nome`	varchar(150)	Não	
`email`	varchar(255)	Não	Único; normalizado (trim e minúsculas) antes de gravar
`telefone`	varchar(20)	Sim	Só dígitos; recebe o prefixo `55` se tiver 10 ou 11 dígitos

endereco

Coluna	Tipo	Nulo	Observações
`id`	integer	Não	PK
`id_morador`	integer	Não	FK para `morador`
`logradouro`	varchar(150)	Não	
`numero`	varchar(20)	Não	Texto livre, para aceitar valores como `12-A` ou `S/N`
`bairro`	varchar(100)	Não	
`municipio`	varchar(100)	Não	
`uf`	varchar(2)	Não	
`cep`	varchar(10)	Não	
`complemento`	varchar(255)	Sim	

material

Coluna	Tipo	Nulo	Observações
`id`	integer	Não	PK
`nome`	varchar(100)	Não	Único; a aplicação também compara ignorando maiúsculas e minúsculas
`descricao`	text	Sim	
`unidade`	varchar(20)	Não	A unidade que pertence ao material (RN02)
`orientacao`	text	Sim	Orientação de preparo e separação
`situacao`	varchar(20)	Não	`ATIVO` ou `INATIVO (Enum SituacaoMaterial)`

pontocoleta

Coluna	Tipo	Nulo	Observações
`id`	integer	Não	PK
`nome`	varchar(255)	Não	
`logradouro`, `numero`, `bairro`, `municipio`, `uf`, `cep`	varchar	Não	Mesmos tamanhos de `endereco, por isso estão resumidos aqui desta forma`

Coluna	Tipo	Nulo	Observações
`complemento`	varchar(255)	Sim	
`contato`	varchar(20)	Sim	Só dígitos, com a mesma normalização do telefone
`horarios`	varchar(255)	Sim	
`situacao`	varchar(20)	Não	`ATIVO` ou `INATIVO` (`Enum SituacaoPonto`)
`recolhimento_domiciliar`	boolean	Não	Indica se o ponto faz recolhimento na casa do morador

operador

Coluna	Tipo	Nulo	Observações
`id`	integer	Não	PK
`id_ponto`	integer	Não	FK para `pontocoleta`
`nome`	varchar(150)	Não	
`email`	varchar(255)	Não	Único; normalizado como o do morador

materialaceito

Coluna	Tipo	Nulo	Observações
`id_ponto`	integer	Não	PK composta e FK para `pontocoleta`
`id_material`	integer	Não	PK composta e FK para `material`
`observacoes`	text	Sim	
`limite_opcional`	numeric	Sim	Limite por solicitação, na unidade do material; deve ser maior que zero (validado no service)

solicitacao

Coluna	Tipo	Nulo	Observações
`id`	integer	Não	PK
`id_ponto`	integer	Não	FK para `pontocoleta`
`id_morador`	integer	Não	FK para `morador`

Coluna	Tipo	Nulo	Observações
`protocolo_unico`	varchar(50)	Não	Único; formato `SC-` + 8 caracteres hexadecimais, gerado no sistema
`copia_logradouro`, `copia_numero`, `copia_bairro`, `copia_municipio`, `copia_uf`, `copia_cep`	varchar	Não	Cópia do endereço no momento da solicitação (RN04)
`copia_complemento`	varchar(255)	Sim	
`situacao`	varchar(50)	Não	Enum `SituacaoSolicitacao`
`data_criacao`	timestamptz	Não	Padrão: momento atual
`data_agendamento`	timestamptz	Sim	
`data_conclusao`	timestamptz	Sim	
`observacoes`	text	Sim	
`versao`	integer	Não	Padrão 0; controle de concorrência (RN06)
`recolhimento_domiciliar`	boolean	Não	Padrão `false`; modalidade escolhida pelo morador

itemsolicitacao

Coluna	Tipo	Nulo	Observações
`id_solicitacao`	integer	Não	PK composta e FK para `solicitacao`
`id_material`	integer	Não	PK composta e FK para `material`
`copia_unidade`	varchar(20)	Não	Cópia da unidade do material (RN04)
`qtd_estimada`	numeric	Não	`CHECK` maior que zero
`qtd_efetiva`	numeric	Sim	`CHECK` maior ou igual a zero; preenchida só na conclusão

historico

Coluna	Tipo	Nulo	Observações
`id`	integer	Não	PK
`id_solicitacao`	integer	Não	FK para `solicitacao`

Coluna	Tipo	Nulo	Observações
`situacao_anterior`	varchar(50)	Sim	Vazia no registro de criação
`situacao_nova`	varchar(50)	Não	
`data`	timestamptz	Não	Padrão: momento atual
`responsavel`	varchar(150)	Não	Nome de quem fez a mudança
`motivo`	text	Sim	Obrigatório na recusa, no cancelamento e no reagendamento (validado no service)
`info_tentativa`	text	Sim	Detalhe da tentativa sem sucesso, no reagendamento
`id_operador`	integer	Sim	FK para `operador`; vazio quando a ação foi do morador

3. Evolução do banco

A estrutura do banco é versionada com o Flyway, e cada alteração é uma migração numerada. Uma migração já aplicada não é editada: um erro é corrigido por uma nova migração. Por isso a sequência abaixo registra também os ajustes de percurso, e o estado final está no diagrama e no script `docs/dbSchema/scriptColetaSeletiva.sql`.

Migração	O que faz	Por quê
V1	Cria o schema `coleta_seletiva` e as nove tabelas, com chaves primárias, estrangeiras, unicidade (e-mail, nome do material, protocolo) e `CHECK` nas quantidades	Estrutura inicial a partir do modelo sugerido na especificação
V2	Troca `morador.telefone` de texto para inteiro	Tentativa com fim de teste, alterando o tipo de telefone para inteiro
V3	Devolve `morador.telefone` para `VARCHAR(20)`	Após o teste, retorna telefone para o seu estado anterior, pois este é o certo
V4	Torna `endereco.id_morador` obrigatório	Cada endereço deve ter um único proprietário (RF02)
V5	Troca `uf` de `CHAR(2)` para `VARCHAR(2)` em `endereco`, `solicitacao` e `pontocoleta`	Alinhar o tipo ao mapeamento do Hibernate, que valida o schema
V6	Torna `recolhimento_domiciliar` e	Um ponto sempre tem situação e sempre

Migração	O que faz	Por quê
	`situacao` obrigatórios em `pontocoleta`	informa se faz recolhimento domiciliar (RF04)
V7	Torna `material.situacao` obrigatória	Todo material é ativo ou inativo (RF03)
V8	Adiciona restrições nomeadas de e-mail único em `morador` e `operador`	Garantir a unicidade do e-mail (RF01)
V9	Remove essas restrições	Elas duplicavam as `UNIQUE` já criadas na V1; a unicidade continua garantida pelas originais
V10	Cria `solicitacao.recolhimento_domiciliar`	Registrar a modalidade escolhida pelo morador, para validar a RN01 contra o que o ponto oferece
V11	Cria `historico.id_operador` com chave estrangeira para `operador`	Identificar quando o responsável é um operador; vazio indica que foi o morador
V12	Converte as datas de `solicitacao` e `historico` para `TIMESTAMPTZ`	Datas de eventos devem incluir fuso ou offset (RNF05)
V13	Reduz `pontocoleta.contato` de 255 para `VARCHAR(20)`	O contato passou a ser tratado como telefone, com o mesmo tamanho do telefone do morador
V14	Carrega dados fictícios de demonstração	Dois pontos, quatro materiais, dois moradores, dois operadores e três solicitações em situações diferentes (RNF06 e RNF07)
V15	Cria índices para os filtros da API	Solicitações por morador, ponto, situação e período; materiais aceitos; histórico; pontos por município e bairro (RNF04)
V16	Renomeia `historico.id_historico` para `id` e torna `operador.id_ponto` obrigatório	Seguir o padrão das demais tabelas e impedir operador sem ponto

Dois casos mostram a vantagem do versionamento. Na V2/V3 e na V8/V9 o erro foi desfeito por uma nova migração, sem alterar nem recriar o banco que já estava em uso.

4. Decisões de projeto

Cada decisão abaixo está descrita com o problema, a escolha e o motivo.

4.1 Flyway para versionar o banco

Problema: o banco precisa ser reproduzível em qualquer máquina, com estrutura e dados de demonstração iguais para toda a equipe e para o avaliador.

Escolha: toda a estrutura e a carga de demonstração ficam em migrações do Flyway (V1 a V16), aplicadas automaticamente na primeira execução. Só o banco coleta_seletiva precisa ser criado à mão.

Motivo: a evolução do banco fica registrada em ordem e cada erro é corrigido por uma nova migração (ver seção 3). O script scriptColetaSeletiva.sql é apenas referência do estado final.

4.2 Credenciais fora do repositório

Problema: a especificação (RNF06) proíbe versionar senhas, tokens ou credenciais de banco.

Escolha: o application.properties lê o usuário e a senha de variáveis de ambiente (USUARIO, com padrão postgres, e SENHA, obrigatória). Para executar na máquina, cada integrante usa uma configuração de execução local do IntelliJ com essas variáveis. A pasta .idea e o arquivo application-local.properties estão no .gitignore.

Motivo: nenhuma credencial entra no histórico do Git, e quem clona o projeto só precisa definir SENHA no properties.

4.3 Hibernate apenas valida o schema

Problema: se o Hibernate criasse ou alterasse tabelas, ele competiria com o Flyway e o banco poderia divergir do que está versionado.

Escolha: spring.jpa.hibernate.ddl-auto=validate. O Flyway é o único responsável pela estrutura.

Motivo: qualquer diferença entre entidade e banco aparece na hora de subir a aplicação, em vez de virar erro em tempo de uso. Foi o que aconteceu na V5 (tipo da UF) e na V16, em que a entidade Historico precisou acompanhar a mudança da coluna.

4.4 Schema próprio no PostgreSQL

Problema: as tabelas não devem ficar misturadas com o schema `public` do banco.

Escolha: todas as tabelas ficam no schema coleta_seletiva, configurado como schema padrão do Hibernate.

Motivo: isola o sistema e deixa claro, nas consultas e nos scripts, a quem cada tabela pertence.

4.5 Responsável do histórico: nome e id_operador opcional

Problema: cada mudança de situação precisa registrar quem a fez, e o responsável pode ser um morador (por exemplo, ao criar ou cancelar) ou um operador.

Escolha: historico.responsavel guarda o nome de quem agiu, e historico.id_operador é uma chave estrangeira opcional para operador. Quando está vazio, o responsável é o morador da solicitação.

Motivo: o campo permite saber diretamente se a ação foi de morador ou de operador, e permite consultar o histórico por operador (GET /historicos?idOperador=). O nome gravado preserva o registro mesmo que o cadastro mude depois.

4.6 Operador pertence a um ponto e não existe sem ele

Problema: o operador só faz sentido vinculado ao ponto em que atua, e o RF07 exige que ele não mexa em solicitações de outro ponto.

Escolha: operador.id_ponto é obrigatório, no banco (V16) e na entidade. Um ponto tem vários operadores, e um operador pertence a um único ponto. Os services verificam se o operador informado pertence ao ponto da solicitação antes de agendar, recusar, iniciar, concluir ou reagendar.

Motivo: impede operador "solto" e mantém o vínculo operador–ponto garantido tanto pelo banco quanto pelas regras.

4.7 Controle de concorrência (RN06)

Problema: a criação com itens e histórico não pode ficar pela metade, duas atualizações simultâneas não podem sobrescrever uma à outra sem aviso.

Escolha: um processo de criação, por exemplo, ocorre em uma única transação, além disso, solicitação tem a coluna versao (@Version): uma atualização com versão desatualizada é rejeitada com 409. Quando só os itens que mudam, a versão é incrementada à força para o conflito ser detectado também nesse caso.

Motivo: garante que uma falha desfaça tudo (rollback) e que nenhuma alteração concorrente se perca em silêncio.

4.8 Normalização de e-mail e telefone

Problema: o mesmo e-mail escrito de formas diferentes burlaria a unicidade, e telefones chegam em formatos variados.

Escolha: o e-mail de morador e operador é normalizado (sem espaços nas pontas e em minúsculas) antes da verificação de duplicidade e da gravação. Telefone e contato do ponto ficam só com dígitos, e recebem o prefixo `55` quando têm 10 ou 11 dígitos. A UF é gravada em maiúsculas.

Motivo: a regra de duplicidade vale de fato (RF01) e os dados ficam em formato consistente.

4.9 Datas com fuso, gravadas em UTC

Problema: a especificação (RNF05) exige que datas de eventos incluam fuso ou offset.

Escolha: as datas de criação, agendamento, conclusão e histórico são TIMESTAMPTZ no banco e OffsetDateTime no Java. O Hibernate trabalha em UTC.

Motivo: o instante do evento fica sem ambiguidade, independentemente de onde o servidor ou o cliente estejam.

4.10 Quantidades em numeric

Problema: a RN02 sugere decimal para peso ou volume e inteiro para unidade de peça.

Escolha: todas as quantidades (qtd_estimada, qtd_efetiva, limite_opcional) usam numeric, e a unidade pertence ao material e é copiada para o item.

Motivo: simplifica o modelo e evita somar quantidades de unidades diferentes no resumo gerencial, que agrupa por material e unidade. A limitação está registrada na seção 5.

4.11 Filtros combináveis com Specification

Problema: as listagens de endereços, pontos de coleta e solicitações aceitam vários filtros opcionais ao mesmo tempo (RF06 e RF09). Por exemplo, as solicitações podem ser filtradas por morador, ponto, situação e período, e os pontos por nome, município, bairro, situação, recolhimento domiciliar e material aceito. Com métodos derivados do repositório (findByXAndY...), seria preciso um método para cada combinação possível, e o número delas cresce rápido a cada filtro novo.

Escolha: EnderecoRepository, PontoColetaRepository e SolicitacaoRepository estendem JpaSpecificationExecutor, e cada filtro é um método estático nas classes EnderecoSpecification, PontoColetaSpecification e SolicitacaoSpecification. O service junta os filtros com Specification.allOf(...) e executa uma única consulta paginada. Cada filtro devolve null quando o parâmetro não foi enviado, e o Spring simplesmente o ignora na combinação. Os detalhes de cada filtro ficam dentro da Specification.

Motivo: um filtro novo vira um método novo, sem multiplicar os métodos do repositório, e a combinação de filtros é feita em um só lugar. Isso mantém a regra de negócio no service (RNF01) e funciona junto com a paginação (RNF04). Os índices da V15 foram criados para os filtros que essas Specifications aplicam, como `lower(municipio)`, `bairro` e `cep`, e a combinação de ponto, situação e período em solicitações.

5. Limitações e próximos passos

Limitações conhecidas

Não há autenticação. O operador é informado em cada requisição, como previsto para o protótipo básico da especificação.

Próximos passos

Interface web responsiva e integração com a API (etapa 3 da disciplina).

Testes automatizados de compatibilidade de materiais, quantidades, transições, limites e concorrência, além de um teste de integração da criação com itens e histórico (RNF08).

Medição de desempenho com a carga de 1.000 solicitações, que já está preparada (RNF04).

6. Arquivos relacionados

README.md: descrição do projeto, tecnologias e instruções de execução.

docs/diagramaER/: diagrama entidade-relacionamento.

docs/dbSchema/scriptColetaSeletiva.sql: estrutura final do banco, apenas para referência.

docs/api/api-docs.json: especificação OpenAPI da API.

docs/dbCarga/: scripts de carga de teste (carga_1000_solicitacoes.sql e limpar_carga.sql).